INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación Gráfica, Visión Computacional y Multimedia				
TÍTULO DE LA PRÁCTICA:	*Transformaciones Geométricas*				
NÚMERO DE PRÁCTICA:	*02*	**AÑO LECTIVO:**	*2026-A*	**NRO. SEMESTRE:**	*IX (noveno)*
FECHA DE PRESENTACIÓN:	*29/04/2026*	**HORA DE PRESENTACIÓN:**	*12:20*		
INTEGRANTE (s): - Riveros Valeriano Sebastian Alfredo				**NOTA:**	
DOCENTE(s): *M. Sc. Ing. Rene Alonso Nieto Valencia*					

SOLUCIÓN Y RESULTADOS

I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS

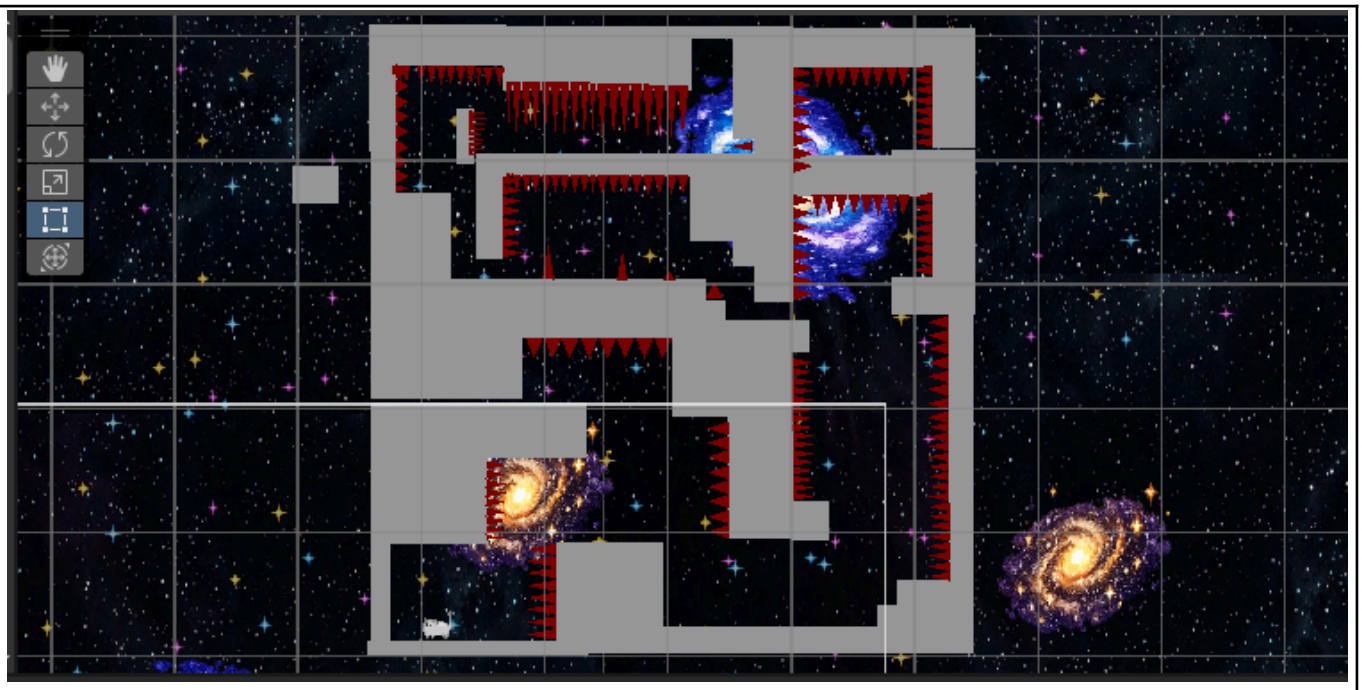
I. Objetivos

- Comprender el uso de transformaciones geométricas 2D.
- Implementar una aplicación interactiva en Unity.
- Aplicar traslación, rotación y escalado en objetos del escenario.
- Utilizar técnicas gráficas como Parallax en un entorno 2D.

II. Descripción del proyecto

Se desarrolló un videojuego 2D en Unity donde el jugador controla un gato que debe atravesar distintos obstáculos hasta llegar a la meta final.
El proyecto utiliza transformaciones geométricas para modificar dinámicamente el entorno y el personaje durante la interacción del usuario.
Las mecánicas principales implementadas fueron:

- Movimiento horizontal del personaje.
- Salto.
- Rotación completa del mapa.
- Escalado dinámico del jugador.
- Sistema de obstáculos y colisiones.
- Sistema de victoria y reinicio.
- Efecto visual Parallax para profundidad de escenari0



III. Desarrollo de la aplicación

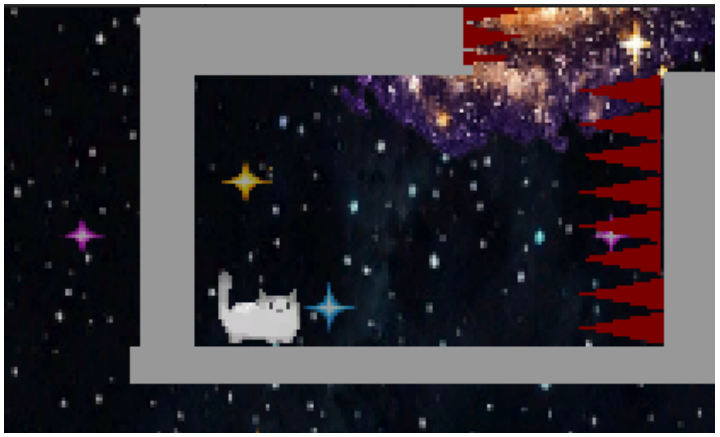
1. Movimiento del personaje (Traslación)

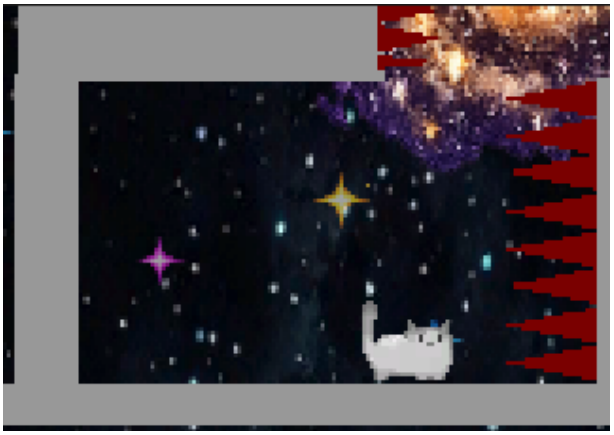
Se implementó un sistema de movimiento horizontal utilizando Rigidbody2D.

Fragmento principal:

```
None
rb.velocity = new Vector2(move * speed, rb.velocity.y);
```

El personaje puede desplazarse a izquierda y derecha mediante teclado.





2. Sistema de salto

El salto fue implementado verificando primero si el personaje se encuentra tocando el suelo.

Fragmento utilizado:

```
None
if ((Input.GetKeyDown(KeyCode.W) || Input.GetKeyDown(KeyCode.UpArrow)) && isGrounded)
{
  rb.velocity = new Vector2(rb.velocity.x, jumpForce);
}
```

Además, la fuerza del salto cambia dependiendo del tamaño actual del personaje.

3. Rotación del mapa

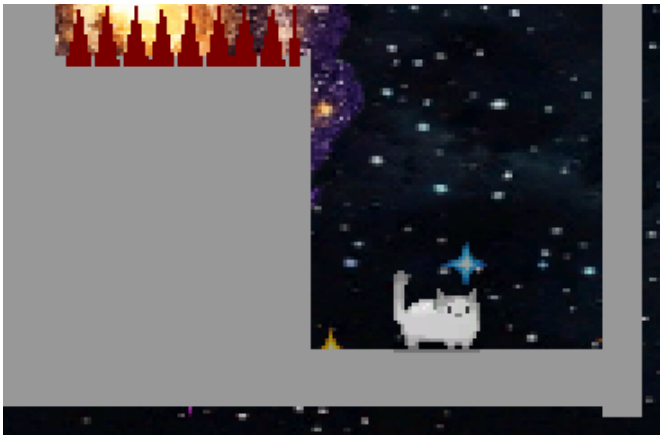
Se implementó una mecánica donde el jugador puede rotar todo el escenario utilizando las teclas Q y E.

La rotación ocurre alrededor de la posición del jugador.

Fragmento principal:

```
None
Quaternion targetRotation = Quaternion.Euler(0, 0, currentRotation);
```

La rotación modifica la orientación del escenario y afecta la forma en la que el jugador supera los obstáculos.



4. Escalado dinámico del personaje

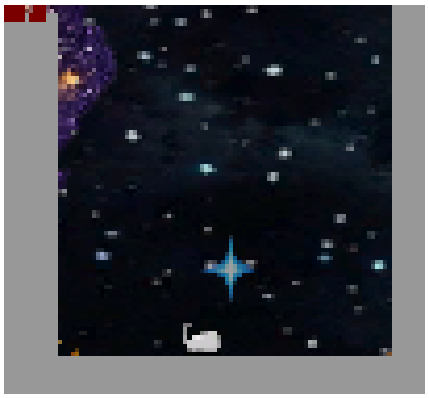
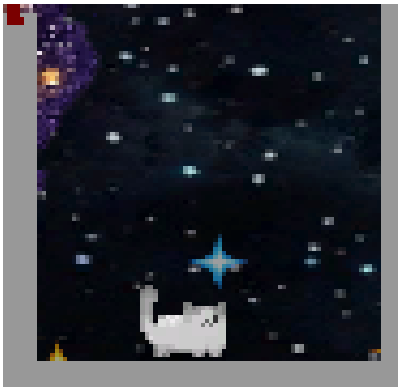
El jugador puede aumentar o disminuir el tamaño del gato utilizando las teclas X y Z.

Fragmento utilizado:

```
None
targetScale = Mathf.Clamp(
  targetScale + scaleStep,
  minScale,
  maxScale
);
```

El cambio de escala afecta:

- tamaño visual
- capacidad de salto
- interacción con obstáculos

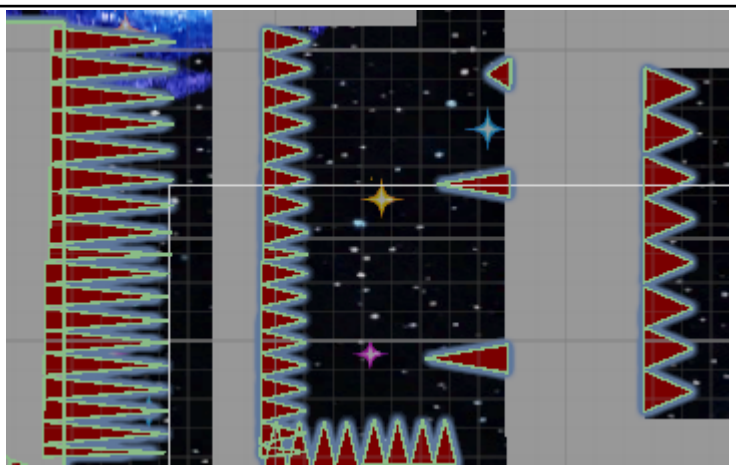



5. Sistema de colisiones

Se agregaron obstáculos tipo espinas que reinician el nivel al colisionar.

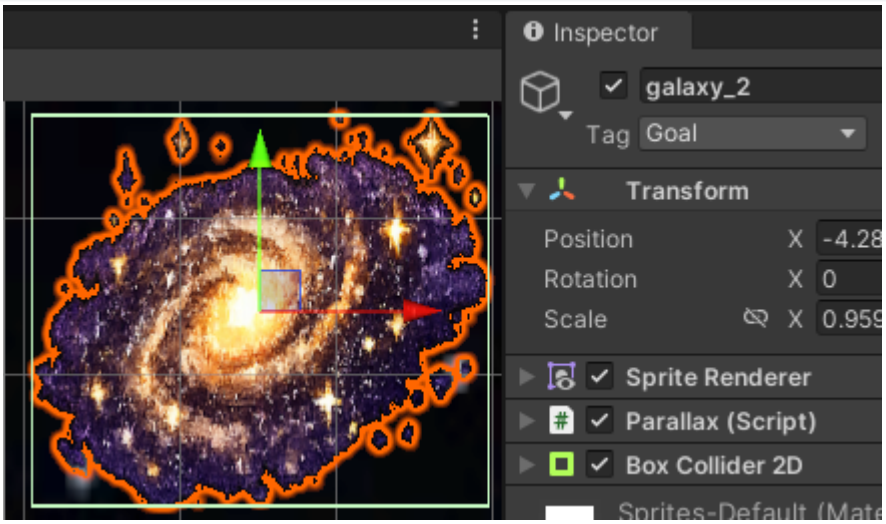
Fragmento utilizado:

```
None
if (collision.CompareTag("Thorn"))
{
  SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
}
```



También se implementó una meta final para completar el escenario.

```
None
if (collision.CompareTag("Goal"))
{
  Time.timeScale = 0f;
}
```



6. Animaciones del personaje

Se implementaron animaciones para:

- correr

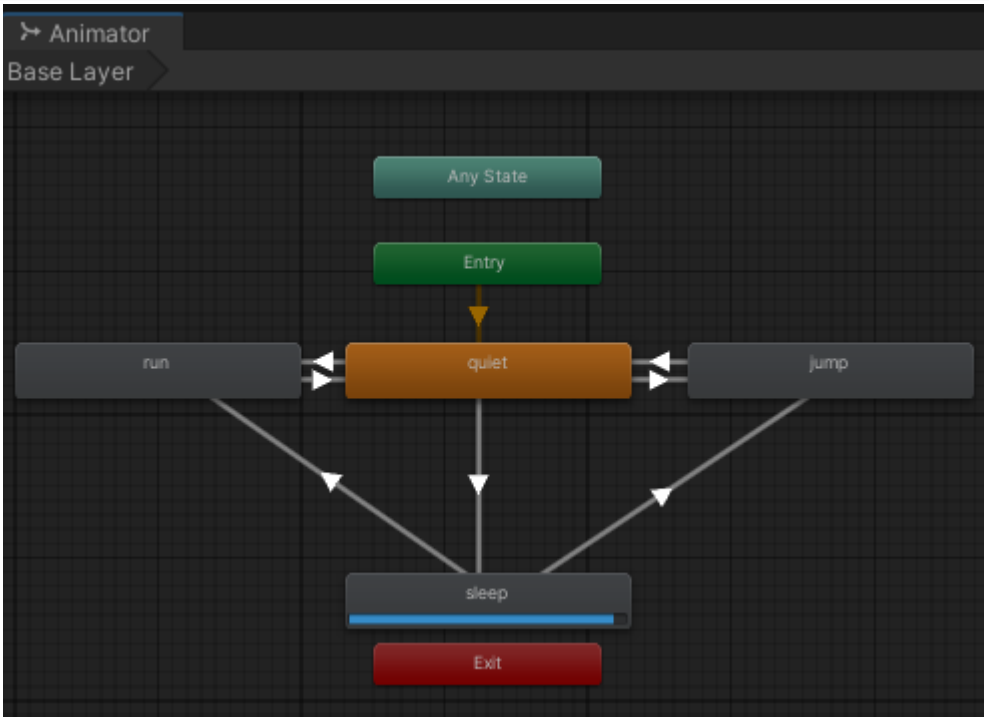
- saltar

- quieto

- dormir

El estado de sueño se activa automáticamente después de un tiempo de inactividad.
Fragmento:

```
None
animator.SetBool("isSleeping", true);
```



7. Implementación de Parallax

Se agregó un sistema Parallax utilizando múltiples capas de fondo:

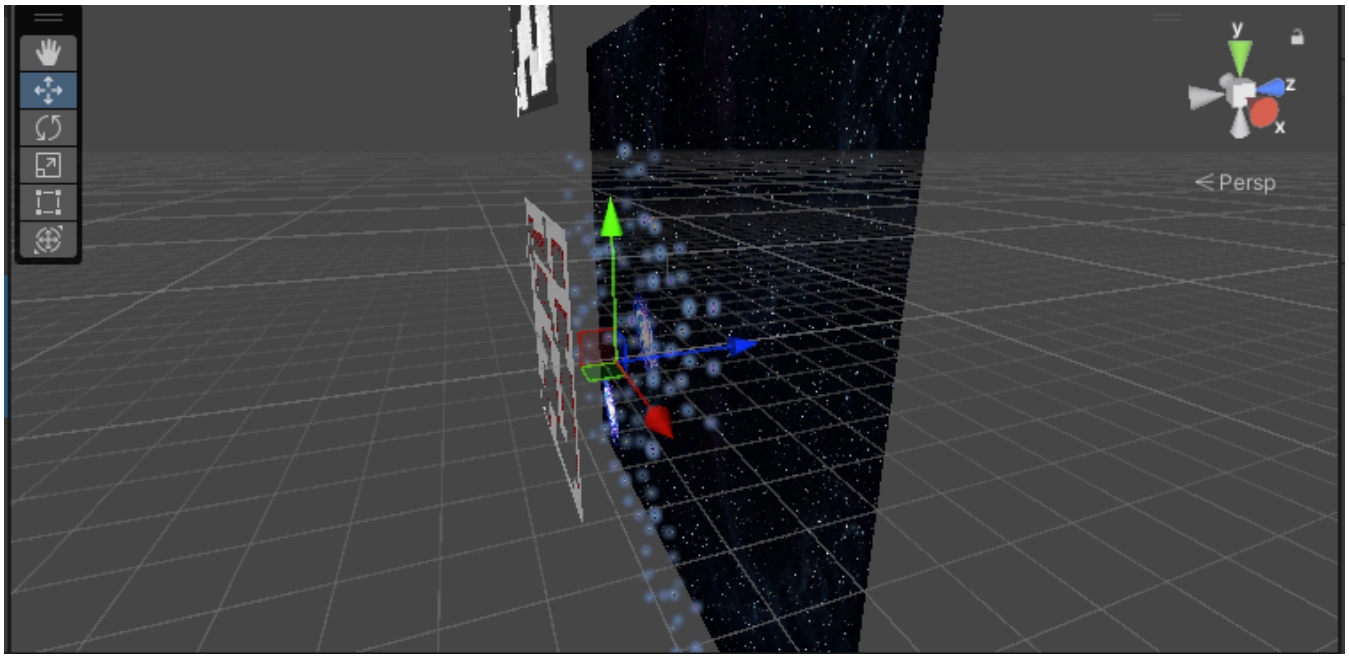
- estrellas
- galaxias
- estrellas cercanas

Cada capa se mueve a distinta velocidad respecto a la cámara para generar sensación de profundidad.

Fragmento principal:

```
None
transform.position += new Vector3(
  deltaMovement.x * parallaxEffect,
  deltaMovement.y * parallaxEffect,
  0f
);
```

Se utilizaron diferentes valores de parallaxEffect según la distancia visual de cada fondo.



IV. Resultados obtenidos

Se logró desarrollar correctamente un videojuego 2D funcional aplicando transformaciones geométricas vistas en laboratorio.

La aplicación implementa:

- traslación
- rotación
- escalado
- colisiones
- animaciones
- Parallax

Las mecánicas desarrolladas permitieron crear un escenario interactivo con diferentes desafíos y efectos visuales.

V. Análisis de resultados

Las transformaciones geométricas permitieron modificar dinámicamente tanto el personaje como el escenario.
La rotación del mapa generó una mecánica distinta para resolver obstáculos, mientras que el escalado afectó directamente la movilidad y el salto del personaje.
El efecto Parallax mejoró la percepción visual de profundidad en el escenario 2D.
Unity facilitó la integración de físicas, animaciones y transformaciones mediante scripts en C#.

VI. Conclusiones

- Se aplicaron correctamente transformaciones geométricas 2D en Unity.
- La traslación, rotación y escalado permitieron crear mecánicas interactivas.
- El efecto Parallax mejoró el aspecto visual del proyecto.
- El uso de scripts en C# permitió controlar dinámicamente el comportamiento del escenario y del personaje.
- Se comprendió la utilidad práctica de las transformaciones geométricas dentro del desarrollo de videojuegos 2D.

VII. Enlace del video

https://drive.google.com/drive/folders/1nkWwEET1tgbLQgETaSVUVwiB2pP1sI32?usp=sharing

II. ANEXOS

Video explicativo: **Video**

Carpeta con códigos del proyecto: **Scripts**

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] Gordon, V. S., & Clevenger, J. L. (2020). Computer Graphics Programming in OpenGL with C++. Stylus Publishing, LLC.

[2] D. P. Kothari, G. Awari, D. Shrimankar , A. Bhende (2019), Mathematics for Computer Graphics and Game Programming: A Self-Teaching Introduction